performance_analysis_p2

Team ID: 30

Algorithm: Iterative PageRank using MapReduce

1. Iteration Time Analysis

We profiled the execution time for a single iteration of the PageRank algorithm across different graph sizes. Since the graph topology remains static and the number of messages (contributions) is constant per iteration, these timings are representative of all 10 iterations.

1.1 Execution Time Breakdown (Per Iteration)

```
cat profile_p2.log

========================================
SLURM_JOB_ID = 58354
SLURM_NODELIST = node[06-07]
SLURM_JOB_GPUS =
========================================
========================================
PROFILING GRAPH SIZE: 10000 Nodes
========================================
Generating 10000 nodes into graph_10000.txt...
----------------------------------------
RESULTS (10000 Nodes)
----------------------------------------
Init Time:    .035338174 s
Map Time:     .265636900 s
Shuffle Time: .165902237 s
Reduce Time:  .146087395 s
Total Iter 1: .612964706 s
----------------------------------------
========================================
PROFILING GRAPH SIZE: 100000 Nodes
========================================
Generating 100000 nodes into graph_100000.txt...
----------------------------------------
RESULTS (100000 Nodes)
----------------------------------------
Init Time:    .394504311 s
Map Time:     1.004431649 s
Shuffle Time: 1.982851588 s
Reduce Time:  1.415070025 s
Total Iter 1: 4.796857573 s
----------------------------------------
========================================
PROFILING GRAPH SIZE: 500000 Nodes
========================================
Generating 500000 nodes into graph_500000.txt...
----------------------------------------
RESULTS (500000 Nodes)
----------------------------------------
Init Time:    1.547493669 s
Map Time:     4.218799383 s
Shuffle Time: 11.719003290 s
Reduce Time:  6.837842879 s
Total Iter 1: 24.323139221 s
----------------------------------------
```

Graph Size (Nodes)	Init Time	Map Time	Shuffle Time	Reduce Time	Total Iteration Time
10,000	0.035 s	0.266 s	0.166 s	0.146 s	**0.613 s**
100,000	0.395 s	1.004 s	1.983 s	1.415 s	**4.797 s**
500,000	1.547 s	4.219 s	11.719 s	6.838 s	**24.323 s**

1.2 Max Graph Size Estimation (10-minute Limit)

- **Time Budget:** 600 seconds.
- **Required Iterations:** 10.
- **Budget per Iteration:** $600/10 = 60$ seconds.
- **Measured Throughput (at 500k nodes):**
  - It takes ≈ 24.3 seconds to process 500,000 nodes (and approx. 5 million edges).
- **Scaling Factor:** The system is dominated by the Shuffle phase ($O(N \log N)$) and Map/Reduce I/O ($O(N + M)$).
- **Estimation:**

$$N_{max} \approx 500,000 \times \frac{60s}{24.3s} \approx \mathbf{1.2} \text{ Million Nodes}$$

(Assuming average degree ≈ 10, this corresponds to approx. 12 Million Edges).

---

2. Scalability Analysis

We analyzed how execution time scales with the number of nodes (N) and edges (M).

2.1 Scaling Behavior

- **Map Phase (Linear Scaling):**
  - $10k \rightarrow 100k$ nodes ($10\times$ increase) resulted in $0.26s \rightarrow 1.00s$ map time ($\approx 3.8\times$ increase). This sub-linear growth at small scales indicates that fixed JVM/Python startup overheads dominate small jobs.
  - $100k \rightarrow 500k$ nodes ($5\times$ increase) resulted in $1.00s \rightarrow 4.22s$ map time ($\approx 4.2\times$ increase). This approaches **pure linear scaling**, confirming that the Map phase is efficient and scales with graph size $O(N + M)$.
- **Shuffle Phase (Super-Linear Scaling):**
  - $100k \rightarrow 500k$ nodes ($5\times$ increase) resulted in $1.98s \rightarrow 11.72s$ shuffle time ($\approx 5.9\times$ increase).
  - This super-linear growth reflects the $O(M \log M)$ complexity of the external sort operation required to group edge contributions by destination node. As the graph grows, the Shuffle phase becomes the primary bottleneck (jumping from 27% of total time at 10k nodes to **48%** at 500k nodes).

2.2 Theoretical vs. Actual Comparison

- **Theoretical Complexity:** $O((N + M) \log(N + M))$ per iteration.

- **Actual Performance:** The measured time closely follows the theoretical curve. The Shuffle phase (Sort) is the dominant term, validating the theoretical expectation that network/disk I/O for sorting is the limiting factor in MapReduce graph processing.

---

3. I/O Performance

In MapReduce PageRank, the "State" (Graph Structure) must be passed through the Map and Reduce phases in every iteration, creating significant I/O overhead.

3.1 Data Volume Calculation

For a graph with N nodes and average degree $d = 10$:

- **Structure Data:** Each node outputs `NodeID + Neighbors`. Approx size: $N \times (8 + 10 \times 8)$ bytes $\approx 88N$ bytes.
- **Contribution Data:** Each edge generates a "Vote". Approx size: $M \times 16$ bytes (DestID + Value).
- **Total Intermediate Data (Per Iteration):**

$$Volume \approx \text{Structure} + \text{Votes} \approx 88N + 16(10N) \approx 250N \text{ bytes}$$

3.2 Measured I/O Overhead

Based on the 500,000 node graph:

- **Estimated Volume:** $500,000 \times 250 \text{ bytes} \approx \mathbf{125} \text{ MB}$ written/read per iteration.
- **I/O Throughput:**
  - The Shuffle + Reduce phases took ≈ 18.5 seconds to process this data.
  - Effective I/O Rate $\approx 125 \text{ MB} / 18.5 \text{ s} \approx \mathbf{6.7} \text{ MB/s}$.
- **Conclusion:** The low effective throughput suggests that the overhead of **text-based serialization** (parsing strings in Python) and **process creation** (MapReduce overhead) is significantly higher than the raw disk bandwidth.

3.3 Network Overhead Estimation

In a fully distributed setup (unlike the single-node profiling):

- The Shuffle phase requires transferring $\approx 125 \text{ MB}$ across the network per iteration.
- On a standard 1Gbps network ($\approx 100 \text{ MB/s}$ practical), network transfer would take ≈ 1.25 seconds.
- Since our measured Shuffle time was 11.72 seconds, the job is **CPU/Disk Bound** (sorting and parsing), not Network Bound. The network would only become the bottleneck for

graphs exceeding 100 Million edges.